

# OCP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

## CHAPTER 17

# Lambda Expressions and Method References

### CHAPTER OVERVIEW

While taking training sessions, I have observed that many Java beginners feel intimidated by lambda expressions. Most of the time this fear is because of the name "lambda".

### EXAM OBJECTIVES COVERED

- ◆ Use lambda expressions implementing functional interfaces

# Chapter 17 Lambda Expressions and Method References

## Exam Objectives

- Use lambda expressions implementing functional interfaces

## 17.1 Understanding Lambda Expressions

### 17.1.1 Lambda Expressions 🙌

While taking training sessions, I have observed that many Java beginners feel intimidated by lambda expressions. Most of the time this fear is because of the name "lambda". The word lambda itself does not create any useful mental picture and that makes it a bit difficult to relate to. On top of that, most books and tutorials start explaining lambda expression as a way to do "functional" programming, which is another buzz word that is hard to relate to when a person is just learning "regular" programming!

Well, I can tell you that there is no need to be afraid. Lambda Expressions are actually very simple. While writing test programs, have you ever been frustrated while typing the words "public static void main(String[] args)" over and over again? I am sure you have been. If you are using an IDE such as NetBeans or Eclipse, then you are probably aware of shortcuts such as `psvm`. You type `psvm`, hit the tab key and the IDE replaces psvm with the text "public static void main(String[] args)" immediately. A lambda expression is just like that but for the compiler. Even though the actual mechanism under the hood is different, you may think of a lambda expression as a shortcut for the compiler that does two things - define a class with a method and instantiate that class. **Conceptually**, as soon as the compiler sees a lambda expression, it expands the expression into a class definition and a statement that instantiates that class. If

you think of it as a tool that saves you from typing a lot of keystrokes, you will start loving it. You will love it so much that you will look for opportunities to use it as much as possible. Let me show you how cool it is.

Imagine that you are working on an application for a car shop. You have a list of cars and you need to filter that list by various criteria. You may need to filter it by company, by price, or by any other property that the users may want. The following code shows how one might do it. You should actually copy all of the following code in a single file named `TestClass.java` so that you can try it out. Just add `import java.util.*;` at the top.

```
record Car(String company, int year, double price, String type){  
    public String toString(){ return "("+company+" "+year+")"; }  
}
```

The `Car` record represents a car with a few properties.

```
class CarMall {  
    List<Car> cars = new ArrayList<>();  
  
    CarMall(){  
        cars.add(new Car("Honda", 2012, 9000.0, "HATCH"));  
        cars.add(new Car("Honda", 2018, 17000.0, "SEDAN"));  
        cars.add(new Car("Toyota", 2014, 19000.0, "SUV"));  
        cars.add(new Car("Ford", 2014, 13000.0, "SPORTS"));  
        cars.add(new Car("Nissan", 2017, 8000.0, "SUV"));  
    }  
  
    List<Car> showCars(CarFilter cf){  
        ArrayList<Car> carsToShow = new ArrayList<>();  
        for(Car c : cars){  
            if(cf.showCar(c)) carsToShow.add(c);  
        }  
    }  
}
```

```

    }
    return carsToShow;
}
}

interface CarFilter{
    boolean showCar(Car c);
}

```

`CarMall` represents the shop. It creates a list of a few `Car` objects. This list contains details of all the cars that the shop has. It has a `showCars` method that returns a list of cars based on any given criteria. Instead of specifying the actual criteria for filtering the cars inside the `showCars` method, it uses a `CarFilter` instance to determine whether a car needs to be listed or not. The `CarFilter` interface declares just the basic structure of a filter. Note that it is not really possible to code the actual filtering criterion inside the `showCars` method because the criterion is determined by the user of the `CarMall` class. By accepting an interface as an argument, the `showCars` method lets the caller decide the criterion.

```

public class TestClass{
    public static void main(String[] args) {
        CarMall cm = new CarMall();
        CarFilter cf = new CompanyFilter("Honda");
        List<Car> carsByCompany = cm.showCars(cf);
        System.out.println(carsByCompany);
    }
}

class CompanyFilter implements CarFilter{
    private String company;
    public CompanyFilter(String c){
        this.company = c;
    }
    public boolean showCar(Car c){

```

```
        return company.equals(c.company());
    }
}
```

`TestClass` represents a third party class that uses `CarMall`. It wants to get the details of all cars from a particular company, say, `Honda`. To do that, it defines a `CompanyFilter` class that contains the actual logic for filtering cars based on company name. At run time, it creates a `CompanyFilter` object and passes it to `CarMall`'s `showCars` method, which returns a filtered list of cars.

Now look at the following code for `TestClass` that uses a lambda expression:

```
public class TestClass{
    public static void main(String[] args) {
        CarMall cm = new CarMall();
        List<Car> carsByCompany = cm.showCars(c → "Honda".equals(c.company()));
        System.out.println(carsByCompany);
    }
}
```

Observe that there is no separate class that implements `CarFilter` and there is no explicit instantiation of a `CarFilter` object either. Both of these tasks have been replaced by a very short statement - `c → c.company.equals("Honda")`. That's it. We have actually eliminated 10 lines of code with that change! You may count them :)

As I said before, the lambda expression used above is just a shortcut for the compiler. For the sake of your understanding, you may imagine that the compiler actually expands this expression into a fully-fledged class definition plus code for instantiating an object of that class (see note). Once you understand how the compiler is able to do this expansion, lambda expressions will seem like a piece of cake to you.

In fact, all the information that is required to do the expansion is available in the context of `cm.showCars( ... )` method call already. The compiler knows that it must pass an object of a class that implements `CarFilter` to the `showCars(CarFilter cf)` method. It knows that this class must implement the `boolean showCar(Car c)` method because that is the only abstract method declared in the `CarFilter` interface. The compiler gathers from the signature of this method that this method receives an argument of type `Car` and returns a `boolean`. From this information, it can easily generate the following code on its own:

```
//assume that this fictional class is created by the compiler.
class XYZ implements CarFilter{
    public boolean showCar(Car <<parameterName>>){
        return <<an expression that returns a boolean must appear here>>;
    }
}
```

The only thing that the compiler cannot generate on its own is the boolean expression that goes inside the `showCar` method. It is not a coincidence that that is exactly what our lambda expression `c → c.company.equals("Honda")` contains. The compiler simply takes the variable name from the left-hand side of `→` of the lambda expression, i.e., `c`, plugs in the expression given on the right-hand side of `→` i.e. `c.company.equals("Honda")`, into the body of the above method and it has the complete code for a class that implements `CarFilter`! Finally, it throws in an instantiation expression `new XYZ();` as a bonus! These are the same two things that are needed to invoke `cm.showCars(CarFilter cf)` method, i.e., code for a class that implements `CarFilter` and an expression that instantiates an object of that class! In fact, before lambda expression were introduced in Java, people used anonymous classes to achieve the same. The following is how you could implement a `CarFilter` the old way:

```
List<Car> carsByCompany = cm.showCars(new CarFilter(){
    public boolean showCar(Car c){
```

```
        return "Honda".equals(c.company());
    }
});
```

The lambda expression used above does the same thing but in lesser number of lines. I suggest you go through the above discussion a couple of times to absorb the steps that the compiler takes for expanding a lambda expression into a fully-fledged class before moving forward. As an exercise, try to expand the lambda expression `x → x.price>10000` into a class that implements `CarFilter`.

Observe that the lambda expression does not specify the method name, the parameter types, and the return type. The compiler infers these three pieces of information from the context in which you put the lambda expression. Thus, a lambda expression must exist within a context that can supply all this information. Furthermore, you know that in Java a method cannot exist on its own. It can exist only as a member of a class (or an interface, for that matter). This implies that the method generated by the compiler for a lambda expression cannot exist on its own either. The compiler must also create a class to put that method in. Since a lambda expression has no name, the class must be such that it requires implementation of exactly one abstract method whose name the compiler can use for the generated method. The only way this can happen is if the class generated by the compiler implements an interface with exactly one abstract method or extends an abstract class with exactly one abstract method. If the interface or the class has more than one abstract method or no abstract method, the compiler wouldn't know which method the code in the lambda expression belongs to.

Java language designers decided not to allow lambda expressions for abstract classes to reduce complexity. Thus, the only question remaining is which interface should the generated class implement. That depends on the context. The type that the context expects is described as the **"target type"** and is the interface that the class generated by the compiler must implement. In the above code, the `showCars` method expects a `CarFilter` object (i.e. object of a class that implements `CarFilter`). Therefore, the generated class must implement `CarFilter`. In technical terms, `CarFilter` is the target type of the lambda expression that is passed as an argument to the `showCars` method.

You may wonder at this point why the compiler can't provide a made up name to the generated method just like it did for the generated class. Well, it could provide a made up name to the method but what would that achieve? Since the programmer wouldn't know that made up name, how would they call this method? Remember that the programmer doesn't need to know the name of the class because the compiler passes an object of this class and the programmer knows the name of the parent class or the interface. You have learnt in the polymorphism section that a reference of type parent class/interface can be used to invoke a method on a subclass. That is exactly what is happening here. The receiver of the lambda expression doesn't care about the type of the actual object that it receives because the receiving code invokes a method on that object using an interface reference. Take a look at the `showCars` method of `CarMall` again. It uses a reference of type `CarFilter` to invoke the `showCar` method. It doesn't matter to this code what name the compiler gives to the generated class. This code only cares about the name of the method.

From the above discussion, it should be clear that a lambda expression can be written only where the target type is an interface with exactly one abstract method. As you saw in the previous chapter, Java has a special name for such an interface: **Functional Interface**. I will get to functional interfaces soon, but first, you need to know the various ways in which you can write a lambda expression.

Due to performance reasons, the compiler does not actually expand lambda expressions into full fledged anonymous classes but it is a good approximation for comprehending lambda expressions quickly. You may google the exact mechanism that Java uses for translating lambda expressions but that is not required for the exam.

### 17.1.2 Parts of a Lambda expression 🙌

You have seen that a lambda expression is basically code for a method in compact



form. It has two parts separated by the "arrow" operator, i.e., `->`. The left side is for variable declarations and the right side is for the code that you want executed. Just like a method, a lambda expression also can have any number of parameters and can return (or not return) a value. Java allows lambda expressions to be written in several different ways. The reason for having so many ways is to cut out as much redundant code as possible. The exam expects you to know all these ways. You will be asked to identify valid and invalid lambda expressions in the exam. From this perspective, I have categorized the variations into two categories - variations on the parameter section and variations on the body section.

There are three possibilities for the parameters section:

1. **No parameter** - If a lambda expression takes no parameters, the parameter part of the expression must have an empty set of parentheses, i.e., `()`. For example,

```
() -> true //valid
```

`-> 1` //invalid, missing variable declaration part

2. **One parameter** - If a lambda expression takes exactly one parameter, the parameter name may be specified within parentheses, i.e., `( pName )` or without the parentheses, i.e., `pName`. For example,

```
a -> a*a //valid  
(a) -> a*a //valid
```

You may also include the parameter type for the parameter name if you want but then you will need to use parentheses. For example,

```
(int a) -> a*a //valid
```

```
int a -> a*a //invalid
```

3. **More than one parameter** - If a lambda expression takes more than one parameter, all the parameter names must be specified within the parentheses, i.e., ( pName1, pName2, pName3 ). For example,

```
(a, b, c) → a + b + c //valid
```

```
a, b → a+b //invalid, parameters must be within ( )
```

Again, parameter types are optional. For example.

```
(int a, int b, int c) → a + b + c //valid
```

If you are specifying parameter types, you must specify them for all the parameters. Thus, (int a, int b, c) → a + b + c would be invalid because it does not specify the type of c

Java 11 has added one more way to declare the parameters. You can use the **var** type for declaring the parameters like this:

```
(var a) → a*a //valid. Observe the parentheses, they are required if you are using var.
```

Normally, there is no need to specify the type for the lambda parameters. However, it is not possible to apply annotations on the parameters unless you specify the type. For example, the following will not compile because annotations cannot be specified directly on a variable: (@NotNull a) → a\*a.

To be able to apply an annotation, you need to have a type for the variable, like this: (@NotNull Integer a) → a\*a . So, if you want to use an annotation on a lambda parameter but don't want to specify the type, you can use the var declaration, like this: (@NotNull var a) → a\*a .

Remember that since a lambda expression essentially provides an implementation of an interface, the rules for implementing an interface discussed in the previous chapter

are equally applicable for lambda expressions. Specifically, the parameter types of a lambda expression (if specified explicitly) must match exactly with the parameter types of the method declared in the interface and the type of the returned value must be covariant with the return type of the interface method.

The syntax of the code part of a lambda expression is simple. It can either be an expression or a block of code contained within curly braces. Given that the body may or may not return a value, there are four possibilities:

1. **Expression with or without a return value** - This is the most common use case and is therefore, the smallest. You can simply put an expression on the right side of `→`. If the expression has a return value and if the lambda expression is supposed to return a value, the compiler will figure out that the value generated by the expression is to be returned and will insert a return statement on its own. You must not write the return keyword. For example,

```
a → a + 2 //valid
```

```
a - > return a + 2 //invalid, must not have return keyword
```

Similarly, an expression that doesn't return any value can also be used directly as the body of the lambda expression. For example,

```
(a, b) → System.out.println(a+b)
```

```
//method call is a valid expression
```

2. **Block of code with or without a return value** - If you have multiple lines of code, you must write them within curly braces, i.e., `{ }`. If the expression is supposed to return a value, you must use a return statement to return the desired value.

This is pretty much the same as writing a method body with or without a return value. You can use this syntax even if you have just one statement in the body. For example, here is a lambda expression that returns a value:

```
2. (a) → {  
    int x = 2;  
    int y = x+a;  
    return y;  
}
```

and here is one that doesn't:

```
() → {  
    int x = 2;  
    int y = 3;  
    System.out.println(x+y);  
}
```

Observe that unlike lambdas with just an expression as their bodies, the statements within the block end with a semi-colon. This is just like a regular code block. All the rules that apply to code within a method body apply to code within a lambda expression's code block as well. After all, the compiler uses this code block to generate a method body.

The OCP exam does not try to trick you with complicated lambda expressions. If you learn the basic rules shown above, you will not have any trouble identifying valid and invalid lambda expressions.

### 17.1.3 Quiz 🙌

**Q 1.** Given:

```
public interface X {  
    Object m(List list);  
}
```

Which of the following options correctly instantiate the interface `X` using a lambda expression?

Select **1** correct option.

- A. `X x = (List l) → l.size();`
- B. `X x = (ArrayList l) → l.size();`
- C. `X x = l → l.toString();`
- D. `X x = l → { };`
- E. `X x = l → l;`

**Correct answer is A, C, E.**

This question tests your knowledge of the rules of implementing an interface. A class that implements `X` must have a method whose parameter type matches exactly with the parameter type of the method in the interface (i.e. `List`) and whose return type is covariant with return type specified in the interface method (i.e. `Object`). Thus, **A** is correct but **B** is not. **A**, **C**, and **E** are correct because `int` (which will be boxed into an `Integer`), `String` and `List` are covariant with `Object` and can be returned from this method. **D** is incorrect because the method `m` is expected to return a value but this lambda expression returns void.

**Q 2.** What will the following code print?

```
public class TestClass {
    static interface Runner{
        void run();
    }
    static interface Walker{
        void walk();
    }
    static class RunnerAndWalker{
        void runOrWalk(Runner r){ r.run(); }
        void runOrWalk(Walker w){ w.walk(); }
```

```

}
public static void main(String[] args) {
    RunnerAndWalker rw = new RunnerAndWalker();
    rw.runOrWalk( ()→{ out.println("walking or running?"); } );
}
}

```

Select **1** correct option.

- A. walking or running?
- B. Nothing will be printed.
- C. Compilation error
- D. Exception at run time

**Correct answer is C.**

This example illustrates the importance of the context in determining the target type of a lambda expression. The `RunnerAndWalker` class has two overloaded `runOrWalk` methods, which take objects of different interface types as arguments. The interfaces have exactly one abstract method that does not take any argument and does not return anything. Other than the names of the interfaces and the names of the methods, the two interfaces are functionally identical.

Now, when the compiler sees the lambda expression passed to the invocation of the `runOrWalk` method, it needs to identify the target type of that expression from the context so that it can invoke the right method. It needs to decide which interface is this lambda expression trying to implement. It checks the number of method parameters, their types, and then the return types of the methods to make a decision. Unfortunately, in this case, all of the three things match. The expression satisfies the needs of both the interfaces equally well. The compiler has no information that can help it break the tie and so, it rejects the code with the following message:

```

error: reference to runOrWalk is ambiguous
    rw.runOrWalk( ()→{ out.println("walking or running?"); } );
        ^

```

```
both method runOrWalk(Runner) in RunnerAndWalker and method runOrWalk(Walker)
in RunnerAndWalker match
```

To fix the error, you may use a cast, like this:

```
rw.runOrWalk( (Runner) ()→{ out.println("walking or running?"); } );
```

### 17.1.4 Using Predicate 🙌

Let's take a look at the `CarFilter` interface that we defined in our `CarMall` example again:

```
interface CarFilter{
    boolean showCar(Car c);
}
```

The whole purpose of this interface is to let you check whether a `Car` satisfies a given criteria so that you could filter a list of Cars. Filtering through a list of objects is such a common requirement in Java application that the Java standard library includes a generic interface for this purpose - `java.util.function.Predicate`. It looks like this:

```
interface Predicate<T>{
    boolean test(T t);
}
```

The `<T>` part means that this interface can be typed for any class as explained in the Generics section of the previous chapter. If the name of that class a `Predicate` is typed to is `T`, then the method `test` will accept an object of type `T` and return a `boolean`.

Let's change the code for `CarMall`'s `showCars` method to use `Predicate` interface:

```

List<Car> showCars(Predicate<Car> cp){
    ArrayList<Car> carsToShow = new ArrayList<>();
    for(Car c : cars){
        if(cp.test(c)) carsToShow.add(c);
    }
    return carsToShow;
}

```

Observe that we have typed `Predicate` to `Car` in the above code. Apart from that, the above code is the same as previous one. But by using the `Predicate` interface instead of writing a custom interface, we have eliminated another three lines of code.

There is no change in the code that calls `showCars` method. The lambda expression that we used earlier, i.e., `cm.showCars(c → c.company.equals("Honda"))` works for this new method as well. It works because the lambda expression never required us to use the name of any interface or method. Therefore, the lambda expression was not tied to a particular interface or method. It was only tied to a particular behavior -, i.e., to a method that takes `Car` as an argument and returns a `boolean`. We relied on the compiler to produce an appropriate class with an appropriate method. We supplied only the raw code for the method body. Earlier the compiler generated a class that implemented the `CarFilter` interface and it now generates a class that implements the `Predicate<Car>` interface using the same code that we wrote in the lambda expression! In fact, the change is so subtle that if you have both the versions of `showCars` method in `CarMall`, the compiler will reject the line `cm.showCars(c → c.company.equals("Honda"))` with the error message, "reference to `showCars` is ambiguous. Both method `showCars(java.util.function.Predicate<Car>)` in `CarMall` and method `showCars(CarFilter)` in `CarMall` match".

I showed only one method in `Predicate` interface but it actually has three `default` methods and one `static` method in addition to the abstract `test` method. I didn't mention them before because they have nothing to do with lambda expressions. You will notice that these methods are basically just helpful utility methods. We haven't



ever seen anyone getting any question on these but, it is better to be aware of these methods.

1. `default Predicate<T> and(Predicate<? super T> other)` : Returns a composed predicate that represents a short-circuiting logical AND of this predicate and another. This is helpful when you have more than one checks to be performed. For example, the following code checks whether a `Car` satisfies two predicates using two separate invocations of `test()` :

```
Predicate<Car> p1 = c → c.company.equals("Honda");
Predicate<Car> p2 = c → c.price>(20000.0);

Car c = ...
if(p1.test(c) && p2.test(c)) System.out.println("yes");
```

Instead of making two calls to `test()` , you could combine the two predicates into one and use only one call to test, like this:

```
Predicate<Car> p3 = p1.and(p2);
Car c = ...
if( p3.test(c) ) System.out.println("yes");
```

The `? super T` part is related to "bounded types" in generics, which I will talk about shortly.

2. `default Predicate<T> negate()` : Returns a predicate that represents the logical negation of this predicate. For example, if you have `Predicate<Car> p = c → c.price<20000;` then `Predicate<Car> notP = p.negate();` represents a predicate where `price is ≥ 20000` .
3. `default Predicate<T> or(Predicate<? super T> other)` : Returns a composed predicate that represents a short-circuiting logical OR of this predicate and another. For example, if you have `Predicate<Car> isHonda`

3. `= c → c.company.equals("Honda");` and `Predicate<Car> isToyota = c → c.company.equals("Toyota");` then `Predicate<Car> isHondaOrToyota = isHonda().or(isToyota);` represents a predicate where the company name is "Honda" or "Toyota".
4. `static <T> Predicate<T> isEqual(Object targetRef)` : Returns a predicate that tests if two arguments are equal according to `Objects.equals(Object, Object)`. This method is a way to convert a class's equals method into a `Predicate`. For example, normally, you would compare two Car objects using `c1.equals(c2)`. You could create a `Predicate` out of the equals method like this - `Predicate equals = Predicate.isEqual(c1);` and then compare `c1` with other `Car` objects using this `Predicate`, i.e., `equals.test(c2)`.

### 17.1.5 Standard functional interfaces in the JDK 🙌

The `Predicate` interface that you saw just now is one among several interfaces defined in `java.util.function` package. The `Predicate` interface captures the use case for testing a condition. In the same way, Java designers have identified several use cases that are encountered in day to day Java development and have created readymade interfaces for them so that every project doesn't have to define the same kind of interfaces. All of them have exactly one abstract method (called as functional method) and are, therefore, functional interfaces. For example, the functional method of the `Predicate` interface is `boolean test(T t)`. Having exactly one abstract method is an important property because it allows you to implement such interfaces using Lambda expressions.

The reason they are called functional interfaces is that they represent a single function without maintaining any state. A functional interface is meant to do exactly one thing. For example, if you take a look at the `Predicate` interface, you will notice that it is meant to check a condition. But the logic for the actual condition is not there in the functional method (because it is an abstract method). The logic is provided by the

code that implements the interface, for example, through a lambda expression. The benefit of this approach is that it allows you to separate the logic from the data upon which that logic is to be applied. That is why it is called "functional programming". Observe that this is exactly opposite to the approach adopted by Object-Oriented Programming, in which an object contains data as well as the logic to operate on that data.

The OCP exam expects you to know about a few standard functional interfaces defined in the `java.util.function` package, namely, `Predicate`, `Consumer`, `Supplier`, `Function`, and `BiFunction`. The exam expects you to know the basic usage of these interfaces without going into the details. You have already seen the usage of the `Predicate` interface. I will talk about the rest now.

### **`java.util.function.Consumer`** 🙌

A `Consumer<T>` is meant to "consume" an object of type `T`. By consume, we mean that we want to do something with the given object. The functional method of the `Consumer` interface is named `accept(T t)`. You just need to write the code for what you want to do with the object passed in this method. For example:

```
//creating a Consumer that consumes a String.  
Consumer<String> strConsumer = s → System.out.println(s.length());  
strConsumer.accept("hello"); //prints 5
```

In the above code, the lambda expression contains only the logic for consuming the String argument. It doesn't contain the actual data on which this logic is to be applied. The actual data comes in the next statement.

### **`java.util.function.Supplier`** 🙌

A `Supplier<T>` supplies an object of type `T` whenever invoked. Its functional method is `get()`. Here is an example that illustrates its usage:

```
public List<Car> getCars(Supplier<Car> carSupplier){
    List<Car> cars = new ArrayList<>();
    for(int i=0; i<10; i++) cars.add(carSupplier.get());
    return cars;
}
```

The above method could be invoked like this:

```
List<Car> cars = getCars(()->new Car()); //assuming Car has a default
constructor.
```

## **java.util.function.Function** 🙌

A `Function<T, R>` takes an argument of type `T`, performs some transformation on that argument, and returns the result of type `R`. The result could be of the same type as that of the argument or of another. Its functional method is `R apply(T t)`. Here is an example that illustrates its usage:

```
public List<String> getCarPropertyList(List<Car> cars, Function<Car,
String> carPropertyFunction){
    List<String> carPropertyList = new ArrayList<>();
    for(Car car : cars) carPropertyList.add(carPropertyFunction.apply(car));
    return carPropertyList;
}
```

The above method takes a list of cars and returns a list containing just a property of the given cars. The `carPropertyFunction` is a "Function" function that takes a `Car` instance and returns a `String`. If you want to get a list of company names of the cars,

for example, you can invoke the above method like this:

```
List<Car> cars = //get a list of cars somehow
List<String> companyNames = getCarPropertyList(cars, c->c.getCompany());//observe that the type of c will be Car
```

Besides the `apply` method, `Function` has two default methods, namely, `andThen` and `compose`. I will not talk about them here because they are not required for the exam. However, it would be a good idea to go through their JavaDoc API descriptions and to try to use them by modifying the above code.

### **java.util.function.BiFunction** 🙌

A `BiFunction<T,U,R>` is similar to (but does not extend) `Function`, except that it takes two arguments of type `T` and `U` respectively, and returns a result of type `R`. Its functional method is `R apply(T t, U u)`. Here is an example that illustrates its usage:

```
public List<Double> getDeliveryCharges(Car car, List<String> cities,
BiFunction<Car, String, Double> costBiF){
    List<Double> deliveryChargesList = new ArrayList<>();
    for(String city : cities) deliveryChargesList.add(costBiF.apply(car, city));
    return deliveryChargesList;
}
```

The above method takes a car and a list of cities and returns the amounts required to deliver that car to each of the given cities. The `costBiF` parameter refers to a `BiFunction` that implements the logic to determine the cost based on the car and the city. The following is a simple implementation that returns a value based on car's price irrespective of the city:

```
List<String> cities = List.of("New York", "Chicago", "Dallas");
Car car = new Car("Honda", 2020, 30000.0, CarType.SUV);
List<Double> deliveryCharges = getDeliveryCharges(car, cities, (kar, city)-
>kar.getPrice()*0.01);//using kar instead of car as the variable name
here, why?
```

Besides the `apply` method, `BiFunction` has one default method, named, `andThen`. `BiFunction` is commonly used for implementing mathematical functions that take two arguments and return the result of the operation. For example, `BiFunction<Double, Double, Double> areaBiF = (a, b)→a*b;`

## java.util.function.UnaryOperator 🙌

A `UnaryOperator<T>` extends `Function<T, T>` and represents an operation on a single operand that produces a result of the same type as its operand. This is a specialization of `Function` for the case where the operand and result are of the same type. Here is an example :

```
UnaryOperator<Integer> flip = i→1*i;
Integer x = 10;
Integer minusX = flip.apply(x);
System.out.println(minusX); //prints -10
```

An important point to note about the functional interfaces described above is that they can be used only with reference types and not with primitive types. So, for example, you cannot do: `BiFunction<double, double, double> areaBiF = (a, b)→a*b;`

## Other functional interfaces 🙌

Besides creating new functional interfaces in the `java.util.function` package,

many existing interfaces in other JDK packages have also been marked with `@FunctionalInterface` annotation because they represent an independent function without being associated with any specific class. Among those, here are the ones that you will encounter in the exam.

1. `java.util.Comparator` - Used while sorting collections.
2. `java.lang.Runnable` - Used while creating new threads.

I will go into their details in relevant chapters.

The JDK standard libraries are littered with interfaces with exactly one abstract method, and technically, they are all functional interfaces but not all of them are annotated with `@FunctionalInterface`. Some of them, such as `java.lang.Comparable`, are not marked with `@FunctionalInterface` because they define intrinsic behaviour of classes and do not define pure independent functionality. So, conceptually, they are not "Functional Interfaces".

### 17.1.6 Using Functional Interfaces with Collections API

Generics made Java collections type safe, while Lambda expressions together with functional interfaces gave them super powers. Most of the classes and interfaces of the collections API were updated in Java 8 to include methods that accept functional interfaces so that programmers can invoke those methods with lambda expressions. Things such as iterating, filtering, and replacing elements, that took several lines of code can now be done by half a line of code.

Let's see a few examples of how functional interfaces and Lists are used together.

#### The `forEach` method

As you have seen before, iterating through a collection is very common requirement.

Prior to Java 8, a common way to iterate through a collection was to use a regular for loop. The `Collection` interface actually extends `java.lang.Iterable` interface and so, it was also possible to use the **enhanced for** loop (aka the **for-each** loop) for this purpose.

With Java 8, a default method named `forEach(Consumer<E> consumer)` was added to the `Iterable` interface, which made it an absolute delight to iterate through the elements of any collection. The following snippet shows the old and the new way to do the same:

```
List<String> list = List.of("a", "b", "c");  
//old way  
for(String s : list){  
    System.out.println(s);  
}  
  
//new way  
list.forEach(s→System.out.println(s));
```

The new way has pretty much taken over completely because of its simplicity.

Similarly, the `Map` interface defines a default `forEach` method that takes a `BiConsumer` instead of `Consumer`. A `BiConsumer` is similar to `Consumer` except that it takes two arguments instead of one. Its functional method is `void accept(T, U)`. Here is an example of how it is used to process the elements of a Map:

```
BiConsumer<String, Integer> bc = (s, i) → System.out.println(s+" is mapped to "+i);  
Map<String, Integer> map = new HashMap<>();  
map.put("One", 1);map.put("Two", 2);  
map.forEach(bc);
```



```
//you could use the lambda expression directly as an argument  
map.forEach((s, i) → System.out.println(s+" is mapped to "+i));
```

You should get familiar with the usage of the `forEach` method because you will see it used on the exam. There are no trick questions in the exam on this method though.

## The `removeIf` method 🙋

Another common operation performed with collections is filtering. The `Collection` interface has a default `removeIf(Predicate<? super E> filter)` method for this purpose. This method removes all of the elements of this list that satisfy the given predicate. Here is a simple example:

```
List<Integer> iList = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));  
Predicate<Integer> p = x→x%2==0;  
iList.removeIf(p);  
System.out.println(iList);
```

The above code prints `[1, 3, 5]`. As you have probably guessed, the lambda expression returns `true` if an element is even. The `removeIf` method executes this lambda expression for each of the elements in the list and removes the element if the expression returns `true` for that element.

Again, we didn't really have to use the variable `p` in the above code. We could have passed the lambda expression directly to the `removeIf` method, i.e., `iList.removeIf( x→x%2==0 );`

## The `sort` method 🙋

A collection has no notion of order but a list does. It makes sense, therefore, that `List` interface has a default `sort(Comparator<? super E> comparator)` method which allows you to sort the elements of this list using the

sorting order determined by the comparator. The `java.util.Comparator` interface has been around since Java 1.2 but it has been made a functional interface in Java 8. Its functional method is `int compare(T o1, T o2)`, which compares its two arguments and returns a negative integer, zero, or a positive integer if the first argument is less than, equal to, or greater than the second. This helps us sort a list in a single line of code as the following example shows:

```
List<String> games = new ArrayList<>(List.of("football", "cricket",  
"baseball", "tennis"));  
games.sort( (a, b)→a.compareTo(b));  
games.forEach(s→System.out.println(s));
```

The output is:

```
baseball  
cricket  
football  
tennis
```

If you want to sort the list in the reverse order, you can change the lambda expression to, `(a, b)→-a.compareTo(b)`. Observe the minus sign in front of `a.compareTo(b)`.

Observe that we are using the `compareTo` method available in the `String` class to compare two strings. Again, if it helps you understand the expression better, you can imagine that the lambda expression will be translated into a definition-cum-instantiation of an anonymous class that implements `Comparator`. Like this:

```
new Comparator<String>(){  
    public int compare(String a, String b){  
        return a.compareTo(b);  
    }  
}
```

### 17.1.7 Scope of variables in a lambda expression 🙌

There is just one more thing that you need to know about lambda expressions. The variables that you define in the variable section of a lambda expression exist in the same scope as which the lambda expression itself exists. This means, you cannot redefine the variables that already exist in that scope. This is illustrated in the following code:

```
List<String> names = Arrays.asList(new String[]{ "alex", "bob", "casy",  
"abel"});  
for(String n : names){  
    Predicate p = n→n.startsWith("a"); //will not compile  
    if(p.test(n)) {  
        System.out.println(n);  
    }  
}
```

The above code will fail to compile with an error message that says, "variable n is already defined". This is because `n` is already in scope within the for loop as well as in the lambda expression. So, when the lambda expression tries to define `n`, the compiler complains.

It is possible to access a variable that is in scope of the lambda expression from within the lambda expression's body but only if that variable is declared as final or is "effectively final". Effectively final means that even though the variable is not explicitly declared as final, its value is not changed though out the scope in which it exists, and so, the compiler assumes it as final. Here is an example:

```
List<String> names = Arrays.asList(new String[]{ "alex", "bob", "casy",  
"abel"});
```

```

int x = 0;
for(String n : names){
    Predicate<String> p = k→{
        System.out.println(n); //valid
        //System.out.println(x); //will not compile
        return k.startsWith("a");
    };
    if(p.test(n)) {
        System.out.println(n);
    }
}
x = 1; //x is being changed here

```

In the above code, the variable `n` is never changed after it is assigned a value. Thus, `n` is effectively final. But `x` is not because it is assigned a new value later in the code. Note that this rule is applicable only for **local variables** and not for instance or static fields. It is possible to access instance and static fields from the body of a lambda expression even if they are not final.

### 17.1.8 Quiz 🙋

Q. Given the following class:

```

public class Item{
    String name; double price;
    public Item(String name, double price){
        this.name = name; this.price = price;
    }
}

```

and the following code appearing in another class

```
double price = 100.0;
Supplier<Item> itemSupplier = ()→ new Item("dummy", price);
price = 200.0;
System.out.println(itemSupplier.get().price);
```

What will be the output?

- A. 100.0
- B. 200.0
- C. Nothing is printed.
- D. Compilation failure.
- E. An exception is thrown at run time.

**Correct answer is D.**

Observe that the variable `price` is not effectively final because its value is being changed in the code. Therefore, it cannot be used in the lambda expression.

## 17.2 Method References

First off, the word reference in "method reference" does not mean the same as in object reference. A method reference does not "refer to" or "point to" a method like a reference variable does to an object because a method is not an object in Java. The word reference here just means indication. As per JLS, a method reference expression is used to refer to the invocation of a method without actually performing the invocation.

If you liked the ease and convenience offered by lambda expressions, you will love

method references because they provide even more opportunity to remove boilerplate code that the compiler can easily infer from the context.

For example, here is a statement that you saw earlier for printing all the elements of a list: `list.forEach( obj → System.out.println(obj) );` It uses a lambda expression to create the `accept` method of the `Consumer` interface. In this lambda, all we are doing is creating a variable named `obj` and then passing it to the `println` method of `System.out`. The variable serves no purpose other than being a conduit to pass an object reference to the method. We don't even care about the name of the variable.

Now, see this statement: `list.forEach(System.out::println);` Here, we are essentially telling the compiler to use the `println` method of `System.out` to generate the code for the `Consumer` object expected by the `forEach` method. The compiler knows that the generated `Consumer`'s `accept` method will be passed an element of the list as an argument. Since `println` also expects one argument, the compiler can easily put two and two together and generate the code that passes the element received by the `accept` method down to the `println` method.

That's pretty much all that method references do. You tell the compiler which method you are interested in using the **double colon** syntax and if the method expects any arguments, the compiler will automatically pick up the values that are available in the context and pass them as arguments to the method that you are interested in. The only catch is that the compiler must be able to figure out exactly which method you want to call **unambiguously** and that it must be possible to call it with the information available in the context, which means the compiler must be able to identity the object on which the method has to be invoked (aka target reference) and the compiler must be able to identity the values that can be passed as arguments to the method.

## Identifying the method

This is easy because the method name is written on the right side of `::` in a method reference expression. For example, in `System.out::println`, `println` is the method name. If you want to create an instance of a class, the fictional method name in that case would be `new`. So, `String::new` would mean that you want to create an

instance of `String`.

## Identifying the target reference

The target reference, i.e., the reference on which the method is to be invoked, is indicated on the left side of `::`. It could be the name of a class (in case of a static method), a variable that is already in scope (in case of an instance method), or a reference that is available in the context in which the method reference expression is written (also in case of an instance method). The first two cases are easy to understand with the following examples:

1. In `Math::random`, `Math` is the class name on which the static method `random` can be invoked. In case of static methods, an instance of the class is not needed to invoke the method and so, only the class name is needed.
2. In `System.out::println`, the target reference is identified by a static variable named `out` of class `System`. It refers to an instance of `PrintStream`. `PrintStream` has an instance method named `println`. Thus, the `out` variable can be used to invoke the `println` method.

In both the examples above, the target reference can be identified clearly at compile time (the `Math` class and the `out` variable in the `System` class) but the compiler can get hold of the target reference from the context as well. Here is an example:

```
List<StringBuilder> sbl = List.of(new StringBuilder("123"));
sbl.forEach(StringBuilder::reverse);
```

Here, `reverse` is an instance method of `StringBuilder`, which means, we need a reference to a `StringBuilder` object on which this method is to be invoked, but there is no clearly identifiable `StringBuilder` reference variable present in the above code! So, how does it work? Well, the compiler is smart to figure out that this method reference expression is present within the context of `List`'s `forEach` method, which takes a `Consumer<StringBuilder>` instance as an argument. The `reverse` method is supposed to be executed inside that `Consumer<StringBuilder>`'s `accept` method. The `accept` method will be

passed a `StringBuilder` reference as an argument. Thus, the context in which the method reference is being used does have a `StringBuilder` reference and it is this `StringBuilder` reference on which the `reverse` method will be invoked. This is, effectively, the same as the following:

```
sbl.forEach( new Consumer<StringBuilder>(){
    public void accept(StringBuilder sb){
        sb.reverse();
    }
});
```

## Identifying the arguments

The values to be passed as arguments is the most confusing part of a method reference because they are never specified explicitly or implicitly anywhere in a method reference expression. The compiler takes all of the available values in the context and passes them to the method as arguments. But for this to happen, the number of values available in the context and their types must match the number of arguments and their types expected by the method. This is illustrated by the following code:

```
Map<String, String> map = new HashMap<>();
map.forEach(System.out::println);
```

`Map<String, String>`'s `forEach` method expects a `BiConsumer<String, String>` and `BiConsumer<String, String>`'s `accept` method gets two arguments (the first argument is the key and the second argument is the value of a key-value pair in the map), which means there are two values in the context of this method reference. But `println` expects only one argument. So, the compiler is not sure which value should be passed down to the `println` method and it rejects the code. You can observe the cause of confusion if you translate this statement to use an anonymous class as follows:



```
map.forEach( new BiConsumer<String, String>(){
    public void accept(String a, String b){
        System.out.println( a or b );//not sure which variable to pass to
        the println method
    }
});
```

Now, see the following example:

```
record Person(String fname, String lname){
    boolean isSpouse(Person anotherPerson){
        System.out.println("Checking "+fname+" "+lname+": "+anotherPerson.lname);
        return lname.equals(anotherPerson.lname);
    }
}
...
Map<Person, Person> map = Map.of(new Person("Amy", "Adams"), new
Person("Sarah", "Adams"));
map.forEach(Person::isSpouse);
```

Can you guess the output?

Observe that `Map<Person, Person>`'s `forEach` expects a `BiConsumer<Person, Person>` and its `accept` method will be invoked with two `Person` references as arguments. Thus, there are two `Person` references in the context of this method reference expression. Now, since `isSpouse` is an instance method of `Person` and it expects one `Person` reference as an argument, the compiler needs two `Person` objects to realize the method reference expression `Person::isSpouse` - one for invoking the `isSpouse` method on, and one for passing as an argument to the `isSpouse` method. Thus, the compiler will use the first `Person` available in the context as the target object and the second `Person` as an argument to the method. This is also evident from the output, which is: `Checking Amy Adams:Adams`. The

value returned by `isSpouse` is not used and so, it is ignored. The following is how you can imagine this statement will be translated as:

```
map.forEach( new BiConsumer<Person, Person>(){
    public void accept(Person a, Person b){
        a.isSpouse(b); //compiler was able to use both of the available arguments
    }
});
```

Now, check out the same example but with a static method:

```
record Person(String fname, String lname){
    public static boolean isSpouse(Person first, Person second){
        System.out.println("Checking "+first.fname+"
"+first.lname+": "+second.lname);
        return first.lname.equals(second.lname);
    }
}
...
Map<Person, Person> map = Map.of(new Person("Amy", "Adams"), new
Person("Sarah", "Adams"));
map.forEach(Person::isSpouse);
```

The above code uses the same method reference but the `isSpouse` method is now **static** and it takes **two** `Person` arguments. As explained above, the context does get two `Person` references (as arguments to `BiConsumer`'s `accept` method). Since `isSpouse` is static, the compiler doesn't need a `Person` reference to invoke the `isSpouse` method. It can invoke the method directly on the `Person` class and can pass the two `Person` references as arguments, like this:

```
map.forEach( new BiConsumer<Person, Person>(){
```

```
public void accept(Person a, Person b){  
    Person.isSpouse(a, b);//compiler was able to use both of the  
available arguments  
}  
});
```

Remember that method references are a mere shortcut for lambda expressions and lambda expressions are a short cut for implementing functional interfaces. So, at the end, a method reference will conceptually be translated to an instance of functional interface. So, if you are ever in doubt whether a particular method reference is valid or not, just check if it correctly implements the functional interface that is being expected at that place.

There are many more ways of using method references than the ones I have described in this section. However, they are rarely used in those ways in practice. Fortunately, the exam doesn't expect you to know about them either. The concepts explained in this section are more than enough for the exam as well as for professional coding.

### 17.2.1 Quiz 🙌

**Q1.** Given the following code:

```
record Person(String fname, String lname) {  
    Person{  
        System.out.println("created "+fname+" "+lname);  
    }  
    boolean isWeird(){  
        return fname.equals(lname);  
    }  
}
```

```

    boolean isSpouse(Person anotherPerson){
        return lname.equals(anotherPerson.lname);
    }
    static void log(Person p){
        System.out.println("Logging "+p.fname());
    }
}
...
Map<String, String> map = Map.of("Tom", "Cruz", "Brad", "Pitt");

```

Identify valid code fragments.

Select **3** correct options.

- A. `map.forEach(Person::new);`
- B. `BiPredicate<Person, Person> spouseChecker = Person::isSpouse;`
- C. `void m(){ System.out::println; }`
- D. `Person p = new Person("a", "b"); Person::log(p);`
- E. `List<Person> l = new ArrayList(); l.removeIf(Person::isWeird);`

**Correct answer is A, B, E.**

In **A**, we are just creating a new `Person` object for each key value pairs in the `Map`. The created `Person` object is ignored/discarded.

In **B**, we are implementing a `BiConsumer` instance using `Person`'s `isSpouse` method. `BiConsumer<Person, Person>`'s `consume` will get two `Person` references, let's call them `p1` and `p2`. The method reference `Person::isSpouse` will be translated to `p1.isSpouse(p2)`.

**C** and **D** are invalid because we are trying to use method references as actual method invocations, which they are not. `System.out.println();` and

`System.out::println;` are not equivalent. The former is an actual method call which will be executed immediately, while the second is just an indication to the compiler that we are talking about the `println` method of `System.out`. Option **D** is invalid for the same reason. We can't pass a parameter to a method reference because we are not invoking the method at this point. In **E**, we are creating a `Predicate<Person>` instance using `Person`'s `isWeird` method.

## 17.3 Exercise

1. Given the following lambda expressions, define appropriate interfaces that can be implemented using these lambda expressions.

`() → true`

`k → k > 5`

2. Given the following interfaces, create lambda expressions that can be used to capture these interfaces.

```
interface Shape{
    double computeArea();
}

interface Operation{
    void operate(String name, double[] params);
}
```

3. Write a method that takes a list of `Image` objects and a `Predicate` as arguments, and returns another list containing only those Images that satisfy the predicate.
4. Assuming that the `Image` class has `width` and `height` properties, invoke the above method that filters out images that are smaller than 100 x 100.